

Document number: P0458R0
Date: 2016-10-09
Project: LEWG
Reply-to: Mikhail Maltsev <maltsevm@gmail.com>

Checking for Existence of an Element in Associative Containers

1. [Abstract](#)
2. [Motivation](#)
3. [Design considerations](#)
 1. [Member function vs free function](#)
 2. [Make iterators convertible to bool](#)
4. [Wording](#)
 1. [Container requirements](#)
 2. [Associative containers](#)
 3. [Unordered associative containers](#)
5. [References](#)

1. Abstract

This paper proposes to add a member function `contains`, which checks, whether or not a given element exists in a container, to standard associative containers.

2. Motivation

The common idiom for checking whether an element exists in an associative container involves doing a lookup and checking the returned iterator:

```
if (some_set.find(element) != some_set.end()) {  
    // ...  
}
```

This idiom suffers from excessive boilerplate code and is inferior to `if (some_set.contains(element))` in terms of expressing intent in code.

It is also not obvious for beginners. Quoting a question from Stack Overflow [\[1\]](#):

How to check if `std::map` contains a key without doing insert?

The only way I have found to check for duplicates is by inserting and checking the `std::pair.second` for false, but the problem is that this still inserts something if the key is unused, whereas what I want is a `map.contains(key)` function.

Another (a little less common) way to perform the same task is using the `count` method. Unfortunately, its use is suboptimal for multisets and multimaps (it has greater complexity than `find`). In some sense it is similar to the `size` method of the standard library containers: it does not replace the `empty` method.

The idea of this proposal is not new. It was discussed on "std-proposals" forum [\[2\]](#), but unfortunately that discussion did not result in a formal proposal.

Qt library includes `QSet` and `QMap` class templates, similar to the standard ones, but they also provide `contains` member function [\[3\]](#).

As Thiago Macieira (one of Qt developers) pointed out in [\[2\]](#), in Qt itself, `contains` method is used much more widely than `find`.

Arguably, for sets (`std::set` and `std::unordered_set`) testing a key for existence is much more common than using the `find` method for any other purpose, yet it has less obvious API.

3. Design considerations

3.1. Member function vs free function

contains could as well be added as a free function. One might even argue, that decoupling of algorithms and containers is one of the core principles of the standard library. Nevertheless, lookup by key is inherent to associative containers, i.e. `find` member function is not equivalent to `std::find`. So, if we were to add `std::contains`, it should work on ranges and be equivalent to `std::find(first, last, element) != last` for consistency.

3.2. Make iterators convertible to bool

Another (non-)option would be to make iterators returned by associative containers contextually convertible to bool, i.e. make

```
if (some_set.find(element))
```

equivalent to

```
if (some_set.find(element) != some_set.end())
```

In some implementations adding such a conversion would be possible without breaking the ABI. For example, this would definitely be possible for unordered associative containers in GCC (libstdc++'s `unordered_set::iterator` consists of a single pointer and for `unordered_set::end` this pointer is `nullptr`). Unfortunately this might not be the case in general, so this option is not proposed

4. Wording

4.1. Container requirements

In [associative.reqmts], "Table 86 — Associative container requirements (in addition to container)", add:

Expression	Return type	Assertion/note pre-/post-condition	Complexity
<code>b.contains(k)</code>	<code>bool</code>	equivalent to <code>b.find(k) != b.end()</code>	logarithmic

In [unord.req], "Table 87 — Unordered associative container requirements (in addition to container)", add:

Expression	Return type	Assertion/note pre-/post-condition	Complexity
<code>b.contains(k)</code>	<code>bool</code>	equivalent to <code>b.find(k) != b.end()</code>	Average case $O(1)$, worst case $O(b.size())$

4.2. Associative containers

In [map.overview], in section "map operations" add:

```
bool contains(const key_type& x) const;
template <class K> bool contains(const K& x) const;
```

In [multimap.overview], in section "map operations" add:

```
bool contains(const key_type& x) const;
template <class K> bool contains(const K& x) const;
```

In [set.overview], in section "set operations" add:

```
bool contains(const key_type& x);
template <class K> bool contains(const K& x) const;
```

In [multiset.overview], in section "set operations" add:

```
bool contains(const key_type& x);
template <class K> bool contains(const K& x) const;
```

4.3. Unordered associative containers

In [unord.map.overview], in section "map operations" add:

```
bool contains(const key_type& k) const;
```

In [unord.multimap.overview], in section "map operations" add:

```
bool contains(const key_type& k) const;
```

In [unord.set.overview], in section "set operations" add:

```
bool contains(const key_type& k) const;
```

In [unord.multiset.overview], in section "set operations" add:

```
bool contains(const key_type& k) const;
```

5. References

1. Stack Overflow question: How to check if std::map contains a key without doing insert?
<http://stackoverflow.com/questions/3886593/how-to-check-if-stdmap-contains-a-key-without-doing-insert>
2. ISO C++ Standard - Future Proposals. Isn't it time we had the bool std::[container]::contains() family of methods?
[https://groups.google.com/a/isocpp.org/forum/#!searchin/std-proposals/set\\$20contains/std-proposals/onDKXlivhkh/dSjfU0onMNUJ](https://groups.google.com/a/isocpp.org/forum/#!searchin/std-proposals/set$20contains/std-proposals/onDKXlivhkh/dSjfU0onMNUJ)
3. Qt Documentation. QSet Class, <http://doc.qt.io/qt-5/qset.html>